

LAPORAN PRAKTIKUM 4

Struktur Data



RAFKI AHMAD PAGAMANDA

2311533016

Kelas D

Implementasi dan Operasi Queue (Antrian)

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

Implementasi dan Operasi Queue (Antrian)

A. Tujuan

Tujuan dari praktikum ini adalah untuk memahami konsep dasar struktur data Queue (antrian) serta cara kerjanya yang menggunakan prinsip FIFO (First In First Out). Selain itu, praktikum ini juga bertujuan untuk mengimplementasikan Queue menggunakan array dan LinkedList, memahami operasi dasar seperti enqueue, dequeue, front, rear, dan display, serta mengkombinasikan Queue dengan struktur data lain seperti Stack untuk melakukan manipulasi data. Mahasiswa juga diharapkan mampu memahami proses iterasi Queue menggunakan Iterator.

B. Teori Singkat

Queue merupakan salah satu struktur data linear yang bekerja dengan prinsip FIFO (First In First Out), dimana elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Operasi utama pada Queue adalah enqueue untuk menambahkan elemen ke bagian belakang dan dequeue untuk menghapus elemen dari bagian depan. Selain itu terdapat operasi lain seperti front untuk melihat elemen terdepan dan rear untuk melihat elemen terakhir tanpa menghapusnya. Queue dapat diimplementasikan menggunakan array maupun LinkedList. Implementasi array biasanya menggunakan konsep circular queue agar lebih efisien dalam penggunaan memori, sedangkan LinkedList lebih fleksibel karena ukurannya dapat berubah secara dinamis. Dalam Java, Queue dapat diimplementasikan menggunakan interface Queue dan kelas seperti LinkedList, serta didukung oleh Iterator untuk melakukan penelusuran data tanpa mengubah isi struktur.

C. Alat dan Bahan

Alat dan bahan yang digunakan dalam praktikum ini meliputi laptop atau komputer sebagai media pengerjaan, software Eclipse IDE sebagai lingkungan pengembangan, bahasa pemrograman Java sebagai alat utama dalam implementasi program, JDK (Java Development Kit) sebagai pendukung eksekusi program, serta GitHub yang digunakan sebagai media penyimpanan dan pengumpulan tugas.

D. Langkah-langkah

1. QueueArray

```
1 package pekan4_2311533016;
2
3 public class QueueArray_2311533016 {
4     int front, rear, size;
5     int capacity;
6     int array[];
7
8     public QueueArray_2311533016(int capacity) {
9         this.capacity = capacity;
10        front = this.size = 0;
11        rear = capacity - 1;
12        array = new int[this.capacity];
13    }
14
15    boolean isFull(QueueArray_2311533016 queue) {
16        return (queue.size == queue.capacity);
17    }
18
19    boolean isEmpty(QueueArray_2311533016 queue) {
20        return (queue.size == 0);
21    }
22
23    void enqueue(int item) {
24        if (isFull(this))
25            return;
26
27        this.rear = (this.rear + 1) % this.capacity;
28        this.array[this.rear] = item;
29        this.size = this.size + 1;
30
31        System.out.println(item + " enqueued to queue");
32    }
33
34    int dequeue() {
35        if (isEmpty(this))
36            return Integer.MIN_VALUE;
37
38        int item = this.array[this.front];
39        this.front = (this.front + 1) % this.capacity;
40        this.size = this.size - 1;
41
42        return item;
43    }
44
45    int front() {
46        if (isEmpty(this))
47            return Integer.MIN_VALUE;
48
49        return this.array[this.front];
50    }
51
52    int rear() {
53        if (isEmpty(this))
54            return Integer.MIN_VALUE;
55
56        return this.array[this.rear];
57    }
58
59    // mencetak elemen antrian
60    void display() {
61        int i;
62        if (front == rear) {
63            System.out.printf("\nAntrian Kosong\n");
64            return;
65        }
66
67        // kunjungi dari belakang dan cetak
68        for (i = front; i < rear; i++) {
69            System.out.printf("%d <-- ", array[i]);
70        }
71        return;
72    }
73 }
```

Class ini digunakan untuk mengimplementasikan struktur data Queue (antrian) menggunakan array. Queue bekerja dengan prinsip FIFO (First In First Out), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Pada class ini terdapat beberapa atribut utama, yaitu front, rear, size, capacity, dan array. Variabel front digunakan untuk menunjukkan posisi elemen paling depan, sedangkan rear menunjukkan posisi elemen terakhir. Variabel size menyimpan jumlah elemen dalam queue, dan capacity menentukan kapasitas maksimum queue.

Method enqueue() digunakan untuk menambahkan elemen ke dalam queue. Proses ini dilakukan dengan menambahkan elemen ke posisi rear dan memperbarui nilai size. Method dequeue() digunakan untuk menghapus elemen dari bagian depan queue, yaitu dari posisi front.

Selain itu, terdapat method isEmpty() dan isFull() untuk mengecek kondisi queue, apakah kosong atau penuh. Method front() dan rear() digunakan untuk melihat nilai elemen terdepan dan terakhir tanpa menghapusnya. Method display() digunakan untuk menampilkan isi queue. Secara keseluruhan, class ini menunjukkan bagaimana queue dapat diimplementasikan secara manual menggunakan array.

2. QueueArrayDriver

```
1 package pekan4_2311533016;
2
3 public class QueueArrayDriver_2311533016 {
4     public static void main(String[] args) {
5
6         QueueArray_2311533016 queue = new QueueArray_2311533016(1000);
7
8         queue.enqueue(10);
9         queue.enqueue(20);
10        queue.enqueue(30);
11        queue.enqueue(40);
12
13        System.out.println("Item di depan " + queue.front());
14        System.out.println("item paling belakang " + queue.rear());
15
16        System.out.println("tampilan queue");
17        queue.display();
18
19        System.out.println();
20
21        System.out.println(queue.dequeue() + " dihapus dari queue");
22
23        System.out.println("item di depan: " + queue.front());
24        System.out.println("item dibelakang: " + queue.rear());
25
26        System.out.println("tampilan queue setelah satu data dihapus");
27        queue.display();
28    }
29 }
```

Class ini berfungsi sebagai driver atau program utama untuk menguji implementasi dari class QueueArray. Di dalam method main(), dibuat sebuah objek queue dengan kapasitas tertentu. Kemudian dilakukan beberapa operasi seperti enqueue() untuk menambahkan elemen ke dalam queue.

Selanjutnya, program menampilkan elemen paling depan menggunakan method front() dan elemen paling belakang menggunakan method rear(). Method display() digunakan untuk menampilkan seluruh isi queue. Program juga melakukan operasi dequeue() untuk menghapus elemen terdepan, kemudian menampilkan kembali isi queue setelah penghapusan tersebut. Class ini membantu pengguna memahami cara kerja queue melalui contoh penggunaan langsung.

3. QueueLinkedList

```
1 package pekan4_2311533016;
2
3 import java.util.LinkedList;
4 import java.util.Queue;
5
6 public class QueueLinkedList_2311533016 {
7     public static void main(String[] args) {
8
9         Queue<Integer> q = new LinkedList<>();
10
11         // tambah elemen (0, 1, 2, 3, 4, 5) ke antrian
12         for (int i = 0; i < 6; i++)
13             q.add(i);
14
15         // Menampilkan isi antrian
16         System.out.println("Elemen Antrian " + q);
17
18         // untuk menghapus kepala antrian
19         int hapus = q.remove();
20         System.out.println("Hapus elemen = " + hapus);
21         System.out.println(q);
22
23         // Untuk melihat antrian terdepan
24         int depan = q.peek();
25         System.out.println("Kepala Antrian = " + depan);
26
27         int banyak = q.size();
28         System.out.println("Size Antrian = " + banyak);
29     }
30 }
```

Class ini mengimplementasikan struktur data queue menggunakan LinkedList dari Java Collection Framework. Pada class ini, digunakan interface Queue dan implementasinya yaitu LinkedList. Elemen ditambahkan menggunakan method add() yang secara otomatis menambahkan elemen ke bagian belakang queue.

Program menampilkan isi queue menggunakan System.out.println(). Untuk menghapus elemen terdepan digunakan method remove(), sedangkan untuk melihat elemen paling depan tanpa menghapus digunakan method peek(). Selain itu, digunakan method size() untuk mengetahui jumlah elemen dalam queue.

Class ini menunjukkan bahwa implementasi queue dapat dilakukan dengan lebih sederhana menggunakan library bawaan Java dibandingkan implementasi manual menggunakan array.

4. ReverseData

```
1 package pekan4_2311533016;
2
3 import java.util.LinkedList;
4 import java.util.Queue;
5 import java.util.Stack;
6
7 public class ReverseData_2311533016 {
8     public static void main(String[] args) {
9
10         Queue<Integer> q = new LinkedList<Integer>();
11         q.add(1);
12         q.add(2);
13         q.add(3); // [1, 2, 3]
14
15         System.out.println("sebelum reverse " + q);
16
17         Stack<Integer> s = new Stack<Integer>();
18
19         while (!q.isEmpty()) { // Q -> S
20             s.push(q.remove());
21         }
22
23         while (!s.isEmpty()) { // S -> Q
24             q.add(s.pop());
25         }
26
27         System.out.println("sesudah reverse = " + q); // [3, 2, 1]
28     }
29 }
```

Class ini bertujuan untuk melakukan pembalikan (reverse) data pada queue dengan bantuan struktur data stack. Pertama, dibuat sebuah queue yang berisi beberapa elemen. Kemudian, elemen-elemen tersebut dipindahkan dari queue ke stack menggunakan method `push()`. Karena stack bekerja dengan prinsip LIFO (Last In First Out), maka urutan elemen akan terbalik.

Setelah itu, elemen dipindahkan kembali dari stack ke queue menggunakan method `pop()`. Hasilnya, urutan elemen dalam queue menjadi terbalik dari semula. Program juga menampilkan isi queue sebelum dan sesudah proses reverse untuk menunjukkan perbedaannya. Class ini memperlihatkan bagaimana dua struktur data (queue dan stack) dapat dikombinasikan untuk menyelesaikan suatu permasalahan.

5. IterasiQueue

```
1 package pekan4_2311533016;
2
3 import java.util.Iterator;
4 import java.util.LinkedList;
5 import java.util.Queue;
6
7 public class IterasiQueue_2311533016 {
8     public static void main(String args[]) {
9
10         Queue<String> q = new LinkedList<>();
11
12         q.add("Praktikum");
13         q.add("Struktur");
14         q.add("Data");
15         q.add("Dan");
16         q.add("Algoritma");
17
18         Iterator<String> iterator = q.iterator();
19
20         while (iterator.hasNext()) {
21             System.out.print(iterator.next() + " ");
22         }
23     }
24 }
```

Pada percobaan ini dilakukan proses iterasi atau penelusuran elemen pada struktur data Queue menggunakan bantuan Iterator. Queue yang digunakan bertipe String dan diimplementasikan dengan kelas `LinkedList` dari Java.

Langkah pertama adalah membuat objek Queue, kemudian menambahkan beberapa elemen berupa kata yaitu "*Praktikum*", "*Struktur*", "*Data*", "*Dan*", dan "*Algoritma*". Setelah itu, digunakan objek Iterator untuk mengakses setiap elemen yang ada di dalam Queue secara berurutan.

Proses iterasi dilakukan menggunakan perulangan `while` dengan kondisi `iterator.hasNext()`, yang berfungsi untuk mengecek apakah masih terdapat elemen berikutnya. Jika masih ada, maka elemen tersebut diambil menggunakan method `iterator.next()` dan langsung ditampilkan ke layar menggunakan `System.out.print()`.

E. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Queue bekerja dengan prinsip FIFO, dimana data yang pertama masuk akan menjadi data pertama yang keluar. Queue dapat diimplementasikan menggunakan array maupun `LinkedList` dengan masing-masing kelebihan, seperti efisiensi memori pada circular array dan fleksibilitas pada `LinkedList`.

Selain itu, Queue juga dapat dikombinasikan dengan struktur data lain seperti Stack untuk melakukan operasi tertentu, seperti membalik urutan data. Proses iterasi menggunakan Iterator juga memudahkan dalam menampilkan isi Queue tanpa mengubah data di dalamnya.

